CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
(CBIT)

PROJECT PROPOSAL

LabSubmit

Programming Laboratory Management & Code Execution Platform

A proposal for institutional adoption as the department's programming lab submission, execution, and evaluation platform

Submitted by: V Pranav
Roll Number: 160125749063
Branch: Computer Science & Engineering — IoT and Cybersecurity including Blockchain Technology
Year: 2nd Year
Institution: Chaitanya Bharathi Institute of Technology (CBIT)
Submitted to: College Administration, CBIT
Date: 31 July 2026

1. Executive Summary

This proposal introduces LabSubmit, main motto to stop students from using “Copy and Paste”. This a full-stack web platform built to manage programming laboratory courses at CBIT — covering roll-number-based student registration, lab assignment authoring, an in-browser multi-language IDE with live code execution, anti-cheat safeguards, and structured evaluation and grading. The platform replaces the current manual, spreadsheet- and file-sharing-driven process of assigning lab tasks, collecting submissions, and recording marks with a single role-based system used by Administrators, Faculty, and Students.

The platform has already been built and is functional end-to-end. This document outlines the problem it solves, its features and architecture, how it would be used across a semester at CBIT, and the case for adopting it as the department's official lab submission system.

2. Introduction & Background

Programming laboratory courses at CBIT require faculty to assign coding tasks, verify that submitted code actually compiles and runs correctly, and record evaluation marks for every student across every lab session. Today, this process commonly relies on a mix of email attachments, pen drives, physical roll-call sheets for tracking, and manual compilation of student code on the lecturer's own machine — a workflow that does not scale well across sections, years, and departments, and offers no protection against copying between students during a live assessment.

LabSubmit was designed and built to directly address these operational gaps using a modern web stack, structured specifically around how CBIT already identifies and organizes students — by 12-digit roll number, branch, year, and section — and is proposed here for evaluation as the department's lab management system.

3. Problem Statement

The current lab assessment workflow at CBIT faces the following gaps:

- **Fragmented submission process —** no centralized system for issuing lab tasks or collecting submissions, relying instead on email or pen-drive handoffs.
- **No registration control —** no enforcement of authorized roll-number ranges during registration, leaving room for unauthorized or duplicate accounts.
- **No built-in code execution —** faculty must manually compile and run each submission on their own machine to verify correctness.
- **No anti-cheat protection —** no safeguards against copy-paste or external code reuse during timed lab assessments.
- **No audit trail —** no audit trail of submission timestamps, evaluation status, or standardized feedback across sections.
- **Manual grading —** marks are recorded manually in spreadsheets, prone to error and with no real-time visibility for students.

4. Objectives

1. Centralize lab task creation, submission, and grading in one role-based platform.
2. Enforce controlled student registration via Admin-authorized roll-number ranges.
3. Provide an in-browser code editor with real multi-language compilation (C, C++, Java, Python, JavaScript), removing the need for manual local verification.
4. Reduce opportunities for academic dishonesty during timed lab assessments through anti-cheat safeguards.
5. Give faculty a structured evaluation workflow — marks, remarks, status, and the ability to reopen a workspace for correction.
6. Give students real-time, transparent access to their grades and feedback.
7. Give administrators system-wide visibility into submissions and grading completion across faculty.

5. Proposed System Overview

LabSubmit is a role-based web application with three access levels, each with a dedicated dashboard:

Role	Login Identity	Primary Responsibility
System Administrator	admin@cbit.in	Faculty accounts, roll-number range allocation, institution-wide reporting
Faculty Lecturer	username@cbit.in	Section & lab management, submission review, grading
Student	12-digit CBIT roll number	Registration, coding, submission, grade viewing

Students register using their 12-digit CBIT roll number against Admin-approved ranges, are assigned to lecturer-managed sections, complete lab assignments in a browser-based IDE, and submit code that locks automatically to preserve submission integrity. Faculty create and grade labs within their own sections; Administrators manage faculty accounts, roll-number ranges, and institution-wide reporting.

6. Key Features

6.1 Administrator

- **Faculty management —** create, edit, and delete lecturer accounts (username@cbit.in), set student capacity limits, and reset passwords.
- **Roll-number allocation —** define authorized roll-number ranges (e.g., 160125000001–160125000120) and assign them to specific lecturers.
- **Institution-wide student directory —** view all registered students across departments, automatically sorted by roll number.
- **System metrics & reporting —** view total submissions, grading completion rates, and performance statistics across faculty.

6.2 Faculty Lecturer

- **Section management —** create, rename, edit, and delete sections for 1st–4th year students (e.g., CSE-1, CIC-1).
- **Lab assessment authoring —** write problem statements, set due dates, and specify input/output constraints for lab tasks.
- **Submission viewer & code runner —** inspect multi-file student submissions, execute code live, and download source files.
- **Evaluation & grading —** award marks (0–100), write feedback remarks, set status (Approved / Rejected / Needs Correction), reopen workspaces for correction, and toggle instant grade publishing.

6.3 Student

- **Controlled self-registration —** registration is validated against a valid 12-digit roll number within an Admin-authorized range, preventing duplicate or unauthorized sign-ups.
- **Monaco online IDE —** a tabbed, multi-file online IDE with syntax highlighting for C, C++, Java, Python, and JavaScript.
- **Anti-cheat protection —** blocks copy, paste, cut, right-click, keyboard shortcuts, and drag-and-drop text transfer during assessments.
- **Single-submission locking —** the workspace becomes read-only immediately after submission unless reopened by the lecturer.
- **Real-time grade view —** published marks, evaluation status, and faculty remarks are visible immediately.

7. System Architecture & Technology Stack

LabSubmit is built as a modern, self-hostable web application using open-source technologies throughout, with no licensing cost to the institution.

Layer	Technology	Purpose
Frontend	Next.js 14 (App Router, TypeScript), Tailwind CSS	User interface for Admin, Faculty, and Student portals
Code Editor	Monaco Editor	In-browser multi-file IDE for C, C++, Java, Python, and JavaScript
Backend / API	Next.js API Routes, custom Node.js server (ws, node-pty)	REST endpoints and real-time terminal/process handling
Database / ORM	Prisma ORM — SQLite (dev) / PostgreSQL (production)	Stores users, sections, labs, submissions, grades
Authentication	JWT (jsonwebtoken), bcryptjs	Role-based session tokens and password hashing
Execution Engine	Native GCC, G++, javac/java, Node.js invocation	Compiles and runs C, C++, Java, Python, and JavaScript submissions
Deployment	Vercel (web + API), Railway (execution engine +	Live cloud deployment; the same

Layer	Technology	Purpose
	PostgreSQL), Docker	Docker image also runs on a college server

8. How LabSubmit Is Used at CBIT

The platform is designed to mirror how a lab course actually runs at CBIT across a semester, without departing from the existing faculty, section, and roll-number structure already used administratively:

1. The Administrator sets up the term — creating faculty accounts for lab instructors and defining the roll-number range for each incoming section or branch (for example, CSE – IoT & Cybersecurity including Blockchain Technology, 2nd year), and assigning that range to the relevant lecturer.
2. Students self-register with their CBIT roll number and a password. Registration is validated live against the range assigned by the Admin, which prevents unauthorized or out-of-range sign-ups.
3. The Lecturer creates sections (for example, “CSE - 2”) and authors lab assignments — problem statements, input/output constraints, and due dates.
4. During the scheduled lab session, students log in, open the assigned lab, and write and test their code in the in-browser IDE, compiling and running it live against the platform's execution engine, with anti-cheat protections active throughout to preserve assessment integrity.
5. Students submit their work; the workspace locks automatically so it cannot be edited after the deadline.
6. The Lecturer reviews submissions in the grading panel, re-runs code where needed, assigns marks and remarks, sets an evaluation status, and publishes grades — or reopens the workspace if a student needs to correct and resubmit.
7. Students see their marks, status, and feedback in real time on their dashboard, without waiting for manually compiled results.
8. At any point, the Administrator can pull system-wide metrics — total submissions and grading completion rate across faculty — for departmental reporting.

In practice, this means a lab session that currently depends on pen drives, email, and a lecturer's personal machine to compile code is instead run entirely through one platform that already reflects CBIT's roll-number and section structure — from registration through to final grade publication.

9. Implementation & Deployment Plan

- **Phase 1 —** Deploy for one section (e.g., CSE – IoT & CSBCT, 2nd year) for a single lab course, on the existing live deployment (Section 12) — requiring no college server, no procurement, and no setup cost.
- **Phase 2 —** Admin account setup, faculty onboarding, and roll-number range configuration for the pilot batch.
- **Phase 3 — Trial:** run 2–3 lab sessions on the platform alongside the existing process, and collect faculty and student feedback.

- **Phase 4 — Rollout:** expand to additional sections and years based on pilot results, moving to the paid hosting tier or to college infrastructure as set out in Section 13.3.
- **Ongoing —** maintenance, support, and long-term continuity are addressed in Section 16. Lab computers require no compilers or software installation of any kind — everything runs in the browser.

10. Benefits to the Institution

- **Operational efficiency —** eliminates email and pen-drive-based submission handling, and removes the need for faculty to manually compile student code.
- **Academic integrity —** built-in safeguards reduce opportunities for copy and paste during timed assessments.
- **Standardized grading —** consistent marks, remarks, and status categories across sections and faculty.
- **Transparency for students —** students see results and feedback immediately instead of waiting on manually compiled spreadsheets.
- **Administrative visibility —** real-time, institution-wide visibility into lab performance and grading completion — useful for outcome-based accreditation reporting (NBA/NAAC).
- **Fits existing structure —** built around CBIT's existing roll-number, branch, and section structure, requiring no change to how students are already identified administratively.

11. Feasibility & Resource Requirements

- **Software —** already built, deployed, and functional — a complete Next.js/TypeScript codebase with a Prisma database schema, Docker configuration, and a live public deployment.
- **Hosting —** already live on cloud hosting (see Section 12). The same Docker image runs unchanged on a college server or VM with GCC, G++, JDK 17+, Python 3, and Node.js installed — see Section 13.3 for the cost comparison.
- **Database —** PostgreSQL in the current deployment, which scales from a single pilot section to multi-department use without further change.
- **Cost —** built entirely on open-source frameworks (Next.js, Prisma, Monaco Editor) — no licensing fees. Infrastructure costs are set out in full in Section 13.
- **Training —** role-based dashboards are self-explanatory; minimal training is needed for Admin, Faculty, or Students to get started.

12. Live Deployment & Access

LabSubmit is not a prototype awaiting funding to be built — it is deployed and reachable now. The committee can open it, log in, and use it before deciding anything.

Item	Detail
Platform URL	https://labsubmit.vercel.app
Application hosting	Vercel — serves the web interface and all REST API endpoints

Item	Detail
Execution engine hosting	Railway — runs the compiler container (GCC, G++, OpenJDK 17, Node.js, Python 3) and the live terminal connection
Database	Managed PostgreSQL (Railway), with all data encrypted in transit
Encryption	HTTPS/TLS enforced on every request; certificates auto-renewed
Source code	github.com/vudigapranav/LabSubmit — available for institutional review
Demonstration access	Reviewer credentials supplied separately to the evaluating committee

The two hosts are split by necessity, not preference: compiling and running student code requires a persistent process with real compilers attached, which the application host does not provide. Students and faculty see one website; the split is invisible to them.

The current deployment runs on free and entry-tier plans, at **no cost to the institution**. It is fully functional for demonstration and for a supervised pilot. Section 13 sets out what it would cost to run at department scale, and Section 15 is candid about what the free tier does not guarantee.

13. Budget & Cost Analysis

LabSubmit carries no licence fee and no development cost — the platform is already built and is contributed to the institution. The only expenditure is infrastructure: somewhere to run it, somewhere to store the data, and a name to reach it by. This section sets out those costs in full, including the ones that are zero, so the committee can see there is nothing hidden.

All figures in Indian Rupees, inclusive of 18% GST where applicable. Foreign-currency hosting converted at ₹95 per USD (August 2026). Cloud pricing is usage-based and may vary by ±10%.

13.1 Phase 1 — Pilot (one section, one semester)

The pilot is deliberately costed to remove any financial barrier to trying the platform. It runs on free and entry-tier hosting on the Vercel-supplied web address, so no procurement, purchase order, or domain request is needed to begin.

Item	Basis	Cost (₹)
Web address	Vercel-issued subdomain (labsubmit.vercel.app)	0
Application hosting	Vercel free tier	0
Execution engine	Railway entry plan, ~₹475/month × 6 months	2,850
Database	PostgreSQL, included in the above	0
SSL certificate	Let's Encrypt, auto-provisioned	0
Software licences	All components open-source	0
Development & setup	Contributed by the student author	0
Total — full pilot semester		**2,850**

Roughly ₹48 per student for a 60-student section, for an entire semester. The pilot can also be run entirely on trial credit at ₹0, accepting the idle-restart delay noted in Section 15.

13.2 Phase 2 — Department-wide deployment (annual)

Once the platform carries graded assessments for a full department (approximately 300 students), it needs paid plans — for guaranteed uptime, for a container that does not sleep between lab sessions, and for licence terms that permit institutional use. This is the figure the committee is actually being asked to approve.

Item	Basis	Annual cost (₹)
Domain name	labsubmit.cbit.ac.in — issued by college IT as a subdomain of the existing institutional domain	0
(alternative)	An independent .in domain, if a college subdomain is not preferred	1,100
SSL / HTTPS certificate	Let's Encrypt, renewed automatically by the host	0
Application hosting	Vercel Pro, 1 seat — ₹1,900/month × 12	22,800
Execution engine hosting	Railway, 1 vCPU / 1 GB always-on container — ~₹2,375/month × 12	28,500
Database (PostgreSQL)	Managed instance, included in the Railway allocation above	0
Automated backups	Nightly database snapshots, 30-day retention	3,000
Uptime monitoring	Alerting if the platform fails during a live lab session	0
Software licences	Next.js, Prisma, PostgreSQL, GCC, OpenJDK — all open-source	0
Development & maintenance	Contributed; no vendor contract required	0
Contingency (15%)	**Compute overage during peak examination weeks**	**8,300**
Total — per year		63,700

Approximately ₹212 per student per year — under ₹18 a month per student for a platform that replaces manual code compilation, pen-drive submissions, and spreadsheet grading across every lab session.

13.3 Three-year projection

Two paths are presented honestly, because the cheaper one is not the one that gets the platform running fastest. **Scenario A** keeps everything on commercial cloud hosting. **Scenario B** pilots on the cloud and then moves onto college infrastructure, which the platform already supports — it ships as a Docker container and runs unchanged on a campus server.

Scenario A — Cloud hosted	Year 1	Year 2	Year 3	3-year total
Scope	Pilot, then 1 dept	Full department	2–3 departments	—
One-time cost	0	0	0	0
Recurring cost	32,000	63,700	78,000	1,73,700
Total	**32,000**	**63,700**	**78,000**	**1,73,700**

Scenario B — Cloud pilot, then college-hosted	Year 1	Year 2	Year 3	3-year total
Scope	Cloud pilot	Migrate to college VM	Steady state	—
One-time cost	0	0 – 75,000	0	0 – 75,000
Recurring cost	2,850	0	0	2,850
Total	**2,850**	**0 – 75,000**	**0**	**2,850 – 77,850**

Scenario B shows ₹0 one-time cost in Year 2 if CBIT already operates a virtual-machine host with spare capacity, which most institutional IT departments do; ₹75,000 covers a dedicated server if none is available.

Recommendation: begin with Scenario A and migrate to Scenario B. Cloud hosting gets the pilot running the same week it is approved, with no procurement and no demand on college IT. Once the platform has proven itself, moving it onto campus infrastructure removes the recurring cost almost entirely, keeps student data physically within the institution, and pays for itself against Scenario A within the second year.

13.4 What the budget deliberately does not include

Several items that normally appear in a software budget are genuinely absent here, and are listed so their absence is understood as accurate rather than overlooked:

- **Licence fees — ₹0.** Every component is open-source under MIT, Apache 2.0, or GPL. There are no per-student seats, no annual renewals, and no vendor lock-in.
- **Development cost — ₹0.** The platform is complete and functional, built as a student project and offered to the institution.
- **Training cost — ₹0.** The three dashboards are self-explanatory; a single one-hour walkthrough per faculty group is sufficient, and can be delivered by the author.
- **Migration cost — ₹0.** There is no legacy system to migrate from — the current process is email, pen drives, and spreadsheets.
- **Hardware for students — ₹0.** LabSubmit runs entirely in a web browser. Existing lab computers need no compilers, no editors, and no software installation of any kind.

That last point is worth weighing on its own: because compilation happens on the server, lab machines no longer need GCC, JDK, or an IDE installed and kept in sync. That is a real reduction in the ongoing configuration workload carried by college IT, and it is not counted as a saving anywhere in the tables above.

14. Data Security, Privacy & Ownership

The platform holds student names, roll numbers, submitted code, and marks. How that data is protected, and who owns it, is set out here.

- **Password storage** — passwords are hashed with bcrypt and are never stored or transmitted in readable form. Nobody, including the administrator, can retrieve a student's password; it can only be reset.

- **Session security** — access is governed by signed JWT tokens carrying the user's role. Every API endpoint re-verifies the role before returning data, so a student cannot reach faculty or administrative functions by manipulating a request.
- **Encryption in transit** — all traffic, including the live terminal connection, runs over TLS. No credential or submission travels unencrypted.
- **Execution authorisation** — before any code is compiled, the server independently verifies the student's identity, that they own the workspace, and that the assessment window is open. The browser is treated as untrusted throughout.
- **Data ownership** — all data belongs to CBIT. There is no third-party analytics, no advertising, no data sharing, and no external processing beyond the hosting providers themselves.
- **Data residency** — under the college-hosted option in Section 13.3, all student data remains on campus infrastructure and never leaves the institution.
- **Backups & retention** — nightly database snapshots with 30-day retention; retention beyond that follows whatever policy the college sets for academic records.
- **Portability** — data lives in a standard PostgreSQL database and can be exported in full at any time. The institution is never locked in.

15. Risks, Limitations & Mitigation

The platform is presented as it actually is, including where it is not yet ready. A pilot is proposed precisely so these limits are tested at small scale rather than discovered during a graded examination.

Risk / limitation	Impact	Mitigation
No isolation between running submissions	Student code executes as an ordinary process in a shared container with no per-student CPU or memory limit. One infinite loop can slow the session for others.	Acceptable under supervision at pilot scale. Per-execution resource limits and container isolation are the first priority before department-wide rollout.
Hosting outage during a live lab session	Students cannot submit within the assessment window.	Uptime monitoring with alerts; the existing offline procedure is retained as a fallback throughout the pilot.
Free-tier hosting has no uptime guarantee	Idle services restart slowly; the licence terms of free plans do not cover institutional use.	Paid plans are budgeted in Section 13.2 before the platform carries any graded assessment.
Anti-cheat is browser-level only	Copy, paste, right-click and shortcuts are blocked in the editor, but a determined student can use a second device.	The platform supplements invigilation, it does not replace it. Fullscreen-exit counts are recorded and flagged to the lecturer.
Single-maintainer dependency	The platform currently has one developer.	Source code, schema, and deployment documentation are handed to the department — see Section 16.
Concurrent load at department scale is untested	Behaviour with 300 simultaneous compilations is not yet measured.	The pilot establishes real load figures before any wider commitment is made.

16. Maintenance, Support & Continuity

A fair question about any student-built system is what happens when the student graduates. It is answered directly here rather than left for the committee to raise.

- **During the pilot** — the author maintains the platform, resolves issues, and supports faculty at no cost to the institution.
- **Documentation handover** — the source code, database schema, deployment guide, and administrator manual are handed to the Department of CSE, so the platform can be operated and modified without the author.
- **Built on standard, widely-taught technology** — Next.js, TypeScript, PostgreSQL, and Docker are mainstream tools. Any competent developer, including a final-year student under faculty supervision, can maintain the codebase.
- **Continuity as a student project** — the platform is well suited to being carried forward by successive project batches under faculty guidance, which both sustains it and gives students real production experience.
- **No vendor lock-in** — nothing proprietary is used. Should the college choose to stop using LabSubmit, the data exports cleanly and no contract has to be exited.
- **Institutional ownership** — the author is willing to transfer the codebase to CBIT outright, so the platform remains the department's asset regardless of who maintains it.

17. Conclusion

LabSubmit is a working, deployable platform that directly addresses the operational gaps in how programming lab courses are currently run at CBIT — from roll-number-controlled registration through to live code execution, anti-cheat protected assessment, and structured grading. It has already been developed and is ready for a pilot deployment within the department.

This proposal requests the college's consideration to trial LabSubmit for an upcoming lab course as a step toward adopting it as the standard platform for programming laboratory management at CBIT.